Package 'ogeom'

October 10, 2026

Type Package

Title Nonlinear Regression on Sparsely Observed Gaussian Curves

Version 0.1.0

Description Implements Galerkin chaos estimators for binary regression on a Gaussian trajectory that is measured at a few irregular times per subject. The package computes the exact conditional features and the information operators of a sampling design, which show which nonlinear effects of the trajectory the design can identify, and fits the estimators with an unbiased risk criterion for the truncation level. Data-generating functions reproduce the simulation designs of the accompanying paper, including Brownian motion measured once at a uniformly distributed time.

License GPL (>= 3)

Encoding UTF-8

Depends R (>= 3.5.0)

Imports stats

Suggests fdapace

NeedsCompilation no

Contents

ogeom-package . . . 2
og_bm_spectral . . . 2
og_design_betas . . . 4
og_design_sampler . . . 5
og_fb_eigen . . . 6
og_fit . . . 7
og_fit_constrained . . . 8
og_kernels . . . 10
og_moments_all . . . 10
og_multi_indices . . . 11
og_predict . . . 12
og_process_from_fpca . . . 13
og_process_kl . . . 14
og_scenario . . . 15
og_simulate . . . 16
og_simulate_bm . . . 17
og_simulation_design . . . 18
og_true_coefficients . . . 19

Index **21**

`ogeom-package` *Nonlinear Regression on Sparsely Observed Gaussian Curves*

Description

Galerkin chaos estimators for binary regression on a Gaussian trajectory measured at a few irregular times per subject, the information operators that show which nonlinear effects of the trajectory a sampling design can identify, and the data-generating processes of the accompanying paper.

Details

Functionals of the trajectory are represented through their Wiener-Ito chaos expansions. For a binary response the law of the data depends on the regression functional g only through the conditional expectation operator K, and under Gaussianity $K^*K = E\{\Gamma(B_D)\}$, where B_D is a random contraction fixed by the design. The information about effects of order q is therefore a tensor moment of B_D, and components of order larger than the largest number of measurements per subject are not identified for Brownian motion and several other processes.

The main functions are `og_fit` and `og_fit_constrained` (estimation), `og_predict` and `og_kernels` (prediction and interpretation), `og_information_profile` (what a design reveals), `og_process_kl`, `og_process_bm` and `og_process_from_fpca` (law of the trajectory), `og_bm_spectral` and `og_bm_augmented` (Brownian motion measured once), and `og_simulate`, `og_simulation_design` and `og_simulate_bm` (simulation designs of the paper).

Author(s)

Ufuk Beyaztas <ufuk.beyaztas@marmara.edu.tr>

`og_bm_spectral` *Fourier-Bessel estimators for Brownian motion measured once*

Description

Estimate the first-order kernel of a binary regression functional of Brownian motion observed once, without error, at a uniformly distributed time.

Usage

```
og_bm_spectral(T, W, Y, J, j = og_bessel_j0_zeros(J))

og_bm_augmented(T, W, Y, J, j_all = og_bessel_j0_zeros(J + 2000))

og_bm_kernel(s, fit)

og_bm_J(n, b)
```

Arguments

T	measurement times.
W	measurements $X(T)$.
Y	binary responses.
J	number of Fourier-Bessel terms.
j	the first J positive zeros of J_0.
j_all	positive zeros of J_0, at least J of them; the remaining ones are used for the tails of the augmented basis.
s	evaluation points in $[0, 1]$.
fit	list returned by og_bm_spectral or og_bm_augmented.
n	sample size.
b	smoothness index of the ellipsoid.

Details

For this design the first-order information operator has kernel $-\log\max(s, u)$, eigenvalues $\lambda_k = 4/j_{0,k}^2$ and Fourier-Bessel eigenfunctions, see og_fb_eigen. The spectral estimator truncates the expansion at J terms and attains the minimax rate $n^{-2b/(2b+3)}$ over Fourier-Bessel ellipsoids with the truncation level given by og_bm_J.

For smoothness $b \geq 1/2$ the ellipsoids impose the boundary condition $f_1(1) = 0$. When the kernel does not vanish at 1 the spectral estimator cannot do better than $n^{-1/4}$; the augmented estimator adds the constant function to the sieve and restores the rate for twice differentiable kernels.

Value

og_bm_spectral and og_bm_augmented return lists with the coefficients theta (and theta_e, the coefficient of the added function, for the augmented estimator), the zeros j and eigenvalues lambda. og_bm_kernel returns the estimated kernel at s; og_bm_J returns $\lceil n^{1/(2b+3)} \rceil$.

Examples

```
## Not run:
## Experiment D, constant kernel (which does not vanish at 1)
set.seed(1)
d <- og_simulate_bm(2000, a = 1, b = 0)
J <- og_bm_J(2000, b = 2)
fit_s <- og_bm_spectral(d$T, d$W, d$Y, J)
fit_a <- og_bm_augmented(d$T, d$W, d$Y, J)
s <- seq(0, 1, length.out = 201)
plot(s, d$f1(s), type = "l", ylim = c(-0.1, 0.4), ylab = "first-order kernel")
lines(s, og_bm_kernel(s, fit_s), col = 2)
lines(s, og_bm_kernel(s, fit_a), col = 4)

## End(Not run)
```

`og_design_betas` *Information operators of a sampling design*

Description

Compute the information blocks $\mathcal{I}_q$ of a design, which determine which components of the regression functional the design identifies and how accurately they can be estimated.

Usage

```
og_design_betas(process, design, sigma2 = 0, nmc = 20000, seed = NULL)

og_gram(beta, alphas, weights = NULL, method = c("auto", "ryser",
        "permutations"))

og_information_profile(process, design, sigma2 = 0, q, nmc = 20000, beta = NULL,
        tol = 1e-06, seed = NULL)
```

Arguments

`process`	process object, see `og_process_kl`.
`design`	design sampler: a function of `n` returning a list of `n` vectors of measurement times, as produced by `og_design_sampler`.
`sigma2`	measurement error variance, or a function of the times returning the error covariance matrix.
`nmc`	number of design draws.
`seed`	optional seed for the design draws; the random number state of the caller is preserved.
`beta`	array of matrices $\beta(D)$, one per design (first dimension), for example the component `beta` of `og_moments_all`.
`alphas`	matrix of multi-indices of one order, see `og_multi_indices`.
`weights`	optional weights of the designs; equal weights by default.
`method`	algorithm for the permanents.
`q`	order of the information block.
`tol`	eigenvalues below `tol` are reported as weak directions.

Details

For a binary response the law of the data depends on the regression functional g only through the conditional expectation operator K. Under Gaussianity, $K^*K = E\{\Gamma(B_D)\}$, the expected second quantization of a random contraction B_D fixed by the design and the measurement error. Its block of order q is the tensor moment $E(B_D^{\otimes q})$; in the basis b_α, its entries are expected permanents of blocks of $\beta(D)$.

`og_design_betas` draws designs from the design law and returns their matrices $\beta(D)$; `og_gram` averages the permanents over such matrices (or over the observed designs); `og_information_profile` combines the two and returns the spectrum. Zero eigenvalues correspond to components that the design cannot identify; for example, components of order q larger than the largest number of measurements per subject are not identified for Brownian motion.

Value

`og_design_betas` returns an `nmc` by L by L array; `og_gram` returns the Gram matrix G_q; `og_information_profile` returns a list with `G`, its eigenvalues `values` and eigenvectors `vectors`, `alphas`, the indices `weak` of eigenvalues below `tol`, and `q`.

Examples

```
## Not run:
## Experiment A: equal linear information, different second-order information
for (d in c("A", "B")) {
  des <- og_simulation_design("A", design = d)
  print(round(og_information_profile(des$process, des$design, des$sigma2, q = 2)$values, 3))
}
## design A: 0.5, 0.5, 0 (the interaction V1 V2 is invisible); design B: 0.5, 0.25, 0.25
## Experiment B: second-order information grows with the share of subjects measured twice
for (p2 in c(0, 0.25, 0.5, 1)) {
  des <- og_simulation_design("B", prob2 = p2)
  ev <- og_information_profile(des$process, des$design, des$sigma2, q = 2, nmc = 5000)$values
  cat(p2, round(min(ev), 4), "\n")
}

## End(Not run)
```

`og_design_sampler` *Design sampler with a random number of uniform measurement times*

Description

Create a design sampler: the number of measurements takes the values in `support` with probabilities `probs`, and given that number the times are independent and uniform on $[0, 1]$.

Usage

```
og_design_sampler(probs, support = seq_along(probs))
```

Arguments

`probs`	probabilities of the numbers of measurements.
`support`	possible numbers of measurements.

Details

Given each number of measurements, the times have a joint law with full support, so the condition under which the null space of the information operators has its explicit diagonal description holds. With such designs, the component of order q of the regression functional is identified for Brownian motion exactly when q does not exceed the largest number of measurements that occurs with positive probability. The times are returned in increasing order.

Value

A function of `n` returning a list of `n` increasing vectors of times.

Examples

```
## Not run:
set.seed(1)
draw <- og_design_sampler(c(0.4, 0.3, 0.2, 0.1))   # Experiment C
draw(3)

## End(Not run)
```

`og_fb_eigen` *Fourier-Bessel eigensystem of the Brownian information operator*

Description

Zeros of the Bessel function J_0, the eigenvalues $4/j_{0,k}^2$ and the eigenfunctions $\varphi_k(s) = J_0(j_{0,k}s^{1/2})/|J_1(j_{0,k})|$.

Usage

```
og_fb_eigen(K)

og_bessel_j0_zeros(K)

og_fb_phi(s, j)

og_bm_dir_fb(L, j = og_bessel_j0_zeros(L))
```

Arguments

`K`	number of zeros or eigenvalues.
`s`	evaluation points in $[0, 1]$.
`j`	zeros of J_0.
`L`	number of directions.

Details

For Brownian motion measured once, without error, at a uniformly distributed time, the first-order information operator is the integral operator with kernel $-\log\max(s, u)$. Its eigenvalues are $4/j_{0,k}^2$, which sum to one, and its eigenfunctions are the Fourier-Bessel functions above. `og_bm_dir_fb` gives these eigenfunctions as directions for `og_process_bm`.

Value

`og_bessel_j0_zeros` returns the first `K` positive zeros; `og_fb_eigen` returns a list with `j` and `lambda`; `og_fb_phi` returns the matrix of eigenfunction values; `og_bm_dir_fb` returns a function of `t`.

Examples

```
## Not run:
e <- og_fb_eigen(5)
e$lambda
sum(og_fb_eigen(2000)$lambda)   # close to 1
s <- seq(0, 1, length.out = 401)
matplot(s, og_fb_phi(s, e$j[1:4]), type = "l", lty = 1)

## End(Not run)
```

`og_fit` *Galerkin chaos estimator*

Description

Estimate the chaos components of a binary regression functional of a sparsely observed Gaussian trajectory.

Usage

```
og_fit(data, Y, process, sigma2 = 0, Q = 2, gram = c("design",
        "conditional", "supplied"), design = NULL, nmc = 20000, beta_design = NULL,
        G_list = NULL, select = c("ure", "full"), J = NULL, tol = 1e-10,
        seed = NULL)

## S3 method for class og_fit
print(x, ...)
```

Arguments

`data`	list with components `t` and `w`: lists of measurement times and centered measurements, one element per subject.
`Y`	binary responses (0 or 1).
`process`	process object, see `og_process_kl`.
`sigma2`	measurement error variance, or a function of the times returning the error covariance matrix.
`Q`	highest chaos order to estimate.
`gram`	source of the Gram matrices: `"design"` uses the known design law through design draws independent of the data; `"conditional"` averages the exact conditional Gram matrices of the observed designs; `"supplied"` uses `G_list`.
`design`	design sampler, needed for `gram = "design"` unless `beta_design` is given.
`nmc`	number of design draws for `gram = "design"`.
`beta_design`	optional array of design matrices $\beta(D)$ from `og_design_betas`, reused across fits.
`G_list`	list of Gram matrices of orders `1:Q`, for `gram = "supplied"`.
`select`	truncation rule: `"ure"` minimizes the risk criterion; `"full"` uses all positive eigen-directions.
`J`	optional vector of fixed truncation levels, overriding `select`.

tol	relative tolerance for positive eigenvalues.
seed	optional seed for the design draws.
x	object of class og_fit.
...	unused.

Details

Write $g = \sum_q g_q$ for the Wiener-Ito chaos decomposition of the regression functional and expand $g_q = \sum_{|\alpha|=q} \theta_\alpha b_\alpha(V)$. For a binary response, $E\{Y\psi_\alpha(O)\} = \langle Kg, Kb_\alpha\rangle$, and since K^*K preserves the chaos decomposition, the coefficients of order q solve $G_q\theta_q = c_q$, free of the components of other orders.

The estimator replaces c_q by its empirical version and inverts G_q on its leading J eigen-directions. Its mean squared error has an exact finite-sample expression, and the truncation level is chosen for each order by an unbiased estimate of the risk up to a constant. Prediction uses $P(Y = 1 \mid O) = \bar{Y} + \sum_q \psi_q(O)^\top \hat{\theta}_q$. Components outside the range of the information operators, for instance those above the identification threshold, are not estimable by any method.

Value

An object of class og_fit: a list with orders (one list per order with the coefficients theta, the truncation level J, the eigenvalues values and eigenvectors vectors of the Gram matrix G, the multi-indices alphas and the risk criterion ure), the mean response Ybar, Q, process, sigma2, gram, the truncation criterion and n.

Examples

```
## Not run:
## Experiment C, interaction scenario
set.seed(1)
des   <- og_simulation_design("C", scenario = "interaction")
train <- og_simulate(1000, des$process, des$design, des$sigma2, des$g, des$r)
test  <- og_simulate(2000, des$process, des$design, des$sigma2, des$g, des$r)
fit <- og_fit(train, train$Y, des$process, des$sigma2, Q = 2, gram = "design", design = des$design)
fit
p <- og_predict(fit, test)
sqrt(mean((p - test$Kg)^2))   # distance to the Bayes predictor P(Y = 1 | O)

## End(Not run)
```

---

og_fit_constrained *Galerkin chaos estimator with bound-constrained truncation*

Description

Fit the Galerkin chaos estimator with truncation levels restricted to fitted components that satisfy $\|\hat{g}_q\|^2 \leq$ bound.

Usage

```
og_fit_constrained(data, Y, process, sigma2 = 0, Q = 2, gram = c("design",
        "conditional", "supplied"), design = NULL, nmc = 20000, beta_design = NULL,
        G_list = NULL, bound = 0.25, tol = 1e-10, seed = NULL)
```

Arguments

data	list with components t and w: lists of measurement times and centered measurements, one element per subject.
Y	binary responses (0 or 1).
process	process object, see og_process_kl.
sigma2	measurement error variance, or a function of the times returning the error covariance matrix.
Q	highest chaos order to estimate.
gram	source of the Gram matrices: "design" uses the known design law through design draws independent of the data; "conditional" averages the exact conditional Gram matrices of the observed designs; "supplied" uses G_list.
design	design sampler, needed for gram = "design" unless beta_design is given.
nmc	number of design draws for gram = "design".
beta_design	optional array of design matrices $\beta(D)$ from og_design_betas, reused across fits.
G_list	list of Gram matrices of orders 1:Q, for gram = "supplied".
bound	upper bound for $\|\hat{g}_q\|^2$.
tol	relative tolerance for positive eigenvalues.
seed	optional seed for the design draws.

Details

Every regression function with values in $[0, 1]$ satisfies $\|g_q\|^2 \leq Var\, g \leq 1/4$, so the default bound adds no assumption. For each order, the truncation level minimizes the risk criterion among the levels whose fitted component satisfies the bound. The restriction protects against eigen-directions dominated by misspecification when the law of the trajectory has been estimated, as in real-data applications.

Value

An object of class og_fit, see og_fit.

Examples

```
## Not run:
## Experiment C, interaction scenario
set.seed(1)
des   <- og_simulation_design("C", scenario = "interaction")
train <- og_simulate(1000, des$process, des$design, des$sigma2, des$g, des$r)
test  <- og_simulate(2000, des$process, des$design, des$sigma2, des$g, des$r)
fit <- og_fit_constrained(train, train$Y, des$process, des$sigma2, Q = 2, design = des$design)
sqrt(mean((og_predict(fit, test) - test$Kg)^2))

## End(Not run)
```

og_kernels *First- and second-order kernels of a fitted regression functional*

Description

Express the fitted first- and second-order components through kernels on the time interval.

Usage

```
og_kernels(fit, grid = seq(0, 1, length.out = 101))
```

Arguments

`fit`	object of class `og_fit` fitted with a process from `og_process_kl`.
`grid`	evaluation grid in $[0, 1]$.

Details

For a process of finite rank, the fitted components satisfy $g_1 = \int \beta_1(t)X(t)\,dt$ and $g_2 = \int\int \beta_2(s,t)\{X(s)X(t) - C(s,t)\}\,ds\,dt$. The kernels show which parts of the trajectory, and which pairs of times, drive the risk. The eigenfunctions are assumed orthonormal in $L^2[0,1]$.

Value

A list with `grid`, the vector `beta1`, and, when `fit$Q >= 2`, the matrix `beta2` and the symmetric coefficient matrix `B`.

Examples

```
## Not run:
## Experiment C, interaction scenario
set.seed(1)
des   <- og_simulation_design("C", scenario = "interaction")
train <- og_simulate(1000, des$process, des$design, des$sigma2, des$g, des$r)
fit <- og_fit(train, train$Y, des$process, des$sigma2, Q = 2, design = des$design)
k <- og_kernels(fit)
image(k$grid, k$grid, k$beta2, xlab = "s", ylab = "t")

## End(Not run)
```

og_moments_all *Conditional moments and exact conditional features*

Description

Compute, for each subject, the conditional means $\hat{U} = E(V \mid O)$ and the matrix $\beta(D)$, and from them the exact conditional features $\psi_\alpha(O) = E\{b_\alpha(V) \mid O\}$.

Usage

```
og_moments_all(data, process, sigma2 = 0, tol = 1e-10)

og_features(mom, alphas)
```

Arguments

data	list with components t and w: lists of measurement times and centered measurements, one element per subject.
process	process object, see og_process_kl.
sigma2	measurement error variance, or a function of the times returning the error covariance matrix.
tol	relative tolerance for the generalized inverse.
mom	list returned by og_moments_all.
alphas	matrix of multi-indices, one per row, as returned by og_multi_indices.

Details

For a design with times $t_1, \dots, t_r$ and measurements $W_j = X(t_j) + \eta_j$, the latent scores are, given O, normal with mean $\hat{U}$ and covariance $I - \beta(D)$, where $\beta_{jl}(D) = \langle u_j, B_D u_l \rangle$. The features $b_\alpha(V) = \prod_l He_{\alpha_l}(V_l)/(\alpha!)^{1/2}$ form an orthonormal basis of the chaos of order $|\alpha|$, and their conditional expectations are Wick polynomials of $\hat{U}$ with covariance $\beta(D)$. These features are exact: no plug-in approximation $b_\alpha(\hat{U})$ is involved.

Value

og_moments_all returns a list with Uhat ($n \times L$) and beta ($n \times L \times L$ array). og_features returns the n by nrow(alphas) matrix of features.

Examples

```
## Not run:
set.seed(1)
des <- og_simulation_design("C")
dat <- og_simulate(200, des$process, des$design, des$sigma2, des$g, des$r, oracle = FALSE)
mom <- og_moments_all(dat, des$process, des$sigma2)
psi <- og_features(mom, og_multi_indices(4, 2))
dim(psi)

## End(Not run)
```

---

og_multi_indices *Multi-indices and Hermite features*

Description

List the multi-indices of a given order and evaluate the normalized Hermite features b_α.

Usage

```
og_multi_indices(L, q)

og_hermite_features(V, alphas)
```

Arguments

L	number of latent scores.
q	order of the chaos.
V	matrix with one row per point and ncol(alphas) columns.
alphas	matrix of multi-indices, one per row.

Details

The multi-indices α with $|\alpha| = q$ index an orthonormal basis $b_\alpha(V) = \prod_l He_{\alpha_l}(V_l)/(\alpha!)^{1/2}$ of the chaos of order q spanned by L latent scores, where He are the probabilists' Hermite polynomials. There are choose(L + q - 1, q) of them.

Value

og_multi_indices returns a matrix with one multi-index per row. og_hermite_features returns the matrix of features $b_\alpha(V)$.

Examples

```
## Not run:
og_multi_indices(2, 2)
og_hermite_features(matrix(c(0.7, -0.3), 1), og_multi_indices(2, 2))

## End(Not run)
```

---

og_predict *Prediction and fitted chaos components*

Description

Predict $P(Y = 1 \mid O)$ for new subjects, and evaluate a fitted chaos component as a function of the latent scores.

Usage

```
og_predict(fit, newdata, clip = TRUE)

og_component(fit, q, V)
```

Arguments

fit	object of class og_fit.
newdata	list with components t and w for the new subjects.
clip	if TRUE, predictions are clipped to $[0, 1]$.
q	order of the component.
V	matrix of latent scores, one row per point.

Details

The prediction is $\bar{Y} + \sum_q \psi_q(O)^\top \hat{\theta}_q$, where $\psi_q(O)$ are the exact conditional features of the new subjects. `og_component` returns $\hat{g}_q(V) = \sum_\alpha \hat{\theta}_\alpha b_\alpha(V)$.

Value

A vector of predicted probabilities, or of component values.

Examples

```
## Not run:
## Experiment C, interaction scenario
set.seed(1)
des   <- og_simulation_design("C", scenario = "interaction")
train <- og_simulate(1000, des$process, des$design, des$sigma2, des$g, des$r)
test  <- og_simulate(2000, des$process, des$design, des$sigma2, des$g, des$r)
fit <- og_fit(train, train$Y, des$process, des$sigma2, Q = 2, design = des$design)
head(og_predict(fit, test))
og_component(fit, 2, cbind(c(1, -1), c(1, 1), 0, 0))   # close to 0.45 exp(-1) V1 V2

## End(Not run)
```

`og_process_from_fpca` *Estimated law of the trajectory for real data*

Description

Turn a principal component analysis through conditional expectation (PACE), as computed by `FPCA` in package **fdapace**, into a process object, and center the measurements with the estimated mean.

Usage

```
og_process_from_fpca(fp, K = fp$selectK, sigma2_min = 0)

og_center_data(data, mu_fun)
```

Arguments

`fp`	object returned by `fdapace::FPCA`.
`K`	number of components; by default the number selected by `FPCA`.
`sigma2_min`	lower bound for the error variance.
`data`	list with components `t` and `w`, the lists of measurement times and measurements of the subjects.
`mu_fun`	mean function of `t`, for example the component `mu_fun` returned by `og_process_from_fpca`.

Details

The theory treats the law of X as known. With real data, the mean, eigenvalues, eigenfunctions and error variance are estimated from the pooled measurements, without the responses, and then treated as known. The signs of the eigenfunctions are fixed so that each has a positive mean. A positive floor for the error variance is advisable when the PACE estimate is close to zero, because a zero error variance makes the conditional features of nearly coincident measurement times unstable.

Value

`og_process_from_fpca` returns a list with components `process` (a process object), `mu_fun`, `sigma2`, `lambda`, `phi_fun` and `sign`. `og_center_data` returns `data` with centered measurements.

Examples

```
## Not run:
library(fdapace)
set.seed(1)
des <- og_simulation_design("C", scenario = "usual")
dat <- og_simulate(500, des$process, des$design, des$sigma2, des$g, des$r, oracle = FALSE)
fp  <- FPCA(dat$w, dat$t, list(dataType = "Sparse", error = TRUE, FVEthreshold = 0.99))
law <- og_process_from_fpca(fp, sigma2_min = 0.01)
cdat <- og_center_data(dat, law$mu_fun)
fit <- og_fit_constrained(cdat, dat$Y, law$process, law$sigma2, Q = 2, gram = "conditional")
fit

## End(Not run)
```

---

og_process_kl *Specification of the Gaussian trajectory*

Description

Construct the process objects used by all estimation functions. A process object describes a centered Gaussian process $X(t) = W(k_t)$ through its covariance function and the cross-covariances between the measurements and the latent scores $V_l = W(u_l)$, $l = 1, \ldots, L$.

Usage

```
og_process_kl(mu, phi, U = diag(length(mu)))

og_process_bm(cumdir)

og_bm_dir_kl(L)

og_bm_dir_const()

og_fourier_phi(K)
```

Arguments

`mu`	positive eigenvalues $\mu_1, \ldots, \mu_K$.
`phi`	function of a vector `t` returning the `length(t)` by `K` matrix of eigenfunction values.
`U`	`K` by `L` matrix with orthonormal columns defining the directions.
`cumdir`	function of `t` returning the `length(t)` by `L` matrix of $\int_0^t u_l(s)\, ds$ for orthonormal directions u_l in $L^2[0,1]$.
`L`	number of directions.
`K`	number of eigenfunctions.

Details

og_process_kl gives a process of finite rank, $X(t) = \sum_k \mu_k^{1/2} \xi_k \phi_k(t)$, with directions given by the orthonormal columns of U; the default U makes the latent scores the standardized principal component scores. og_process_bm gives standard Brownian motion, for which the Gaussian Hilbert space is $L^2[0,1]$, $k_t = 1_{[0,t]}$ and $W(h) = \int h\, dX$; the directions are specified through their primitives.

og_bm_dir_kl, og_bm_dir_const and og_fourier_phi supply standard choices: the Karhunen-Loeve directions of Brownian motion, the constant direction (for which $W(u) = X(1)$), and Fourier eigenfunctions, the first two of which are those of the rotating process used in the paper.

Value

og_process_kl and og_process_bm return a list with components L (number of directions), cov (covariance function of two time vectors) and cross (function returning the matrix of $Cov\{W(u_l), X(t_j)\}$), together with the defining quantities. The other functions return functions of t.

Examples

```
## Not run:
## rotating process of Experiment A
phi <- function(t) cbind(sqrt(2) * cos(2 * pi * t), sqrt(2) * sin(2 * pi * t))
rot <- og_process_kl(c(0.5, 0.5), phi)
rot$cov(c(0, 0.25), c(0, 0.25))
## rank-four Fourier process of Experiment C
procC <- og_process_kl(c(1, 0.6, 0.35, 0.2), og_fourier_phi(4))
## Brownian motion with its first three Karhunen-Loeve directions
bm <- og_process_bm(og_bm_dir_kl(3))
bm$cross(c(0.2, 0.7))

## End(Not run)
```

---

og_scenario *Regression functions of the simulations*

Description

Regression functions used in the paper's simulations, as functions of the latent scores.

Usage

```
og_scenario(scenario = c("interaction", "usual"), delta = 0.45,
        slope = c(1.5, -1, 0.5))
```

Arguments

scenario	"interaction" or "usual".
delta	amplitude of the interaction.
slope	slopes of the logistic index.

Details

`"interaction"`: $g = 1/2 + \delta \sin(V_1)\sin(V_2)$. It has no linear component, and its component of order two is $\delta e^{-1} V_1 V_2$, so it is invisible to linear methods and identified only when subjects with at least two measurements occur. `"usual"`: the single-index logistic model $g = \mathrm{logit}^{-1}(\sum_l s_l V_l)$, dominated by its linear component.

Value

A list with `g`, a function of a matrix of latent scores, and `r`, the number of scores it uses.

Examples

```
## Not run:
sc <- og_scenario("interaction")
sc$g(matrix(c(1, 1), 1))

## End(Not run)
```

`og_simulate` *Simulate sparse functional data with a binary response*

Description

Generate measurement times, noisy measurements of a Gaussian trajectory and binary responses from the exact joint Gaussian law, without discretizing the trajectory.

Usage

```
og_simulate(n, process, design, sigma2 = 0, gfun, r, oracle = TRUE,
        nq = 20)
```

Arguments

`n`	number of subjects.
`process`	process object, see `og_process_kl`.
`design`	design sampler, see `og_design_sampler`.
`sigma2`	measurement error variance, or a function of the times returning the error covariance matrix.
`gfun`	regression function of an $n \times r$ matrix of latent scores, with values in $[0, 1]$.
`r`	number of latent scores on which `gfun` depends.
`oracle`	if `TRUE`, also compute $P(Y = 1 \mid O)$.
`nq`	number of quadrature nodes per coordinate.

Details

The latent scores $V = (W(u_1), \ldots, W(u_L))$ are drawn from $N(0, I_L)$, and given V the values of the trajectory at the design times are drawn from their conditional Gaussian law. Measurement errors are added, and Y is drawn with $P(Y = 1 \mid X) = g(V_1, \ldots, V_r)$. Optionally, the Bayes predictor $P(Y = 1 \mid O)$ is computed by Gauss-Hermite quadrature for evaluation. The settings of the paper's simulations are available through `og_simulation_design`.

Value

A list with `t` and `w` (lists of times and measurements), `Y`, the latent scores `V`, `g` (the values of the regression function) and, if `oracle = TRUE`, `Kg`.

Examples

```
## Not run:
set.seed(1)
des <- og_simulation_design("C", scenario = "interaction")
dat <- og_simulate(1000, des$process, des$design, des$sigma2, des$g, des$r)
table(lengths(dat$t))   # numbers of measurements per subject
mean(dat$Y)

## End(Not run)
```

---

og_simulate_bm *Brownian motion measured once at a uniform time (Experiment D)*

Description

Generate the data of Experiment D: one noiseless measurement $W = X(T)$ of standard Brownian motion at a uniform time T, and a binary response with $P(Y = 1 \mid X) = 1/2 + \delta \sin W(u)$.

Usage

```
og_simulate_bm(n, a = 1, b = 0, delta = 0.4)
```

Arguments

`n`	number of subjects.
`a`	coefficient of the constant part of the direction.
`b`	coefficient of $1 - s$.
`delta`	amplitude of the regression.

Details

The direction u is proportional to $a + b(1 - s)$, and the first-order kernel of the regression functional is $\delta e^{-1/2} u$. The kernel does not vanish at $s = 1$ unless $a = 0$, which is the situation in which the augmented estimator is needed.

Value

A list with `T`, `W`, `Y`, the latent score `Vu`, the values `g` of the regression function and the true first-order kernel `f1`, a function of s.

Examples

```
## Not run:
set.seed(1)
d <- og_simulate_bm(2000, a = 0, b = 1)   # kernel vanishing at 1
fit <- og_bm_spectral(d$T, d$W, d$Y, og_bm_J(2000, b = 2))
s <- seq(0, 1, length.out = 201)
plot(s, d$f1(s), type = "l")
lines(s, og_bm_kernel(s, fit), col = 2)

## End(Not run)
```

`og_simulation_design` *Simulation designs of the paper*

Description

Settings of Experiments A, B and C of the paper: the process, the design sampler, the error variance and the regression function, ready for `og_simulate`.

Usage

```
og_simulation_design(experiment = c("A", "B", "C"), design = c("A", "B"),
        prob2 = 0.5, scenario = c("interaction", "usual"), delta = 0.45,
        sign = 1)
```

Arguments

`experiment`	`"A"`, `"B"` or `"C"`.
`design`	design of Experiment A.
`prob2`	probability of a second measurement in Experiment B.
`scenario`	regression of Experiment C, see `og_scenario`.
`delta`	amplitude of the interaction.
`sign`	sign of the interaction in Experiments A and B.

Details

Experiment A uses the rotating process $X(t) = Z_1 \cos(2\pi t) + Z_2 \sin(2\pi t)$, one noiseless measurement and the interaction regression. Under design A the time is uniform on $\{0, 1/4\}$ and the interaction is invisible; under design B it is uniform on $[0, 1]$ and the interaction is identified, although both designs carry the same linear information.

Experiment B uses Brownian motion with two orthonormal contrasts supported on $[0, 1/2]$ and $[1/2, 1]$, one or two measurements per subject with probability `prob2` of a second one, and error variance 0.05. The interaction of the contrasts is identified only through subjects measured twice.

Experiment C uses a rank-four process with Fourier eigenfunctions and eigenvalues 1, 0.6, 0.35 and 0.2, between one and four measurements with probabilities 0.4, 0.3, 0.2 and 0.1, error variance 0.1, and the regression of `og_scenario`. Experiment D is generated by `og_simulate_bm`.

Value

A list with `process`, `design`, `sigma2`, `g` and `r`.

Examples

```
## Not run:
set.seed(1)
## Experiment A: the interaction is invisible under design A and estimable under design B
for (d in c("A", "B")) {
  des <- og_simulation_design("A", design = d)
  dat <- og_simulate(2000, des$process, des$design, des$sigma2, des$g, des$r)
  fit <- og_fit(dat, dat$Y, des$process, des$sigma2, Q = 2, design = des$design)
  print(round(fit$orders[[2]]$theta, 3))   # V1*V2: 0 under design A, near 0.166 under design B
}
## settings of Experiments B and C
desB <- og_simulation_design("B", prob2 = 0.5)
desC <- og_simulation_design("C", scenario = "usual")

## End(Not run)
```

---

og_true_coefficients *True chaos coefficients and the Bayes predictor*

Description

Compute, for simulated data, the true chaos coefficients $\theta_\alpha = E\{g(V)b_\alpha(V)\}$ and the Bayes predictor $P(Y = 1 \mid O) = E\{g(V) \mid O\}$.

Usage

```
og_true_coefficients(gfun, r, alphas, nq = 30)

og_oracle_Kg(mom, gfun, r, nq = 20)
```

Arguments

gfun	regression function of a matrix of latent scores.
r	number of latent scores on which gfun depends.
alphas	matrix of multi-indices, see og_multi_indices.
nq	number of quadrature nodes per coordinate.
mom	list returned by og_moments_all.

Details

Both quantities are computed by tensor Gauss-Hermite quadrature. Given O, the latent scores are normal with mean $\hat{U}$ and covariance $I - \beta(D)$, so the Bayes predictor is a Gaussian integral of g. They serve to evaluate estimators in simulations.

Value

og_true_coefficients returns the named vector of coefficients; og_oracle_Kg returns the vector of probabilities.

Examples

```
## Not run:
sc <- og_scenario("interaction")
og_true_coefficients(sc$g, 2, og_multi_indices(2, 2))   # V1*V2: 0.45 exp(-1)

## End(Not run)
```

Index

∗ **package**
    ogeom-package, 2

og_bessel_j0_zeros *(*og_fb_eigen*)*, 6
og_bm_augmented, *2*
og_bm_augmented *(*og_bm_spectral*)*, 2
og_bm_dir_const *(*og_process_kl*)*, 14
og_bm_dir_fb *(*og_fb_eigen*)*, 6
og_bm_dir_kl *(*og_process_kl*)*, 14
og_bm_J *(*og_bm_spectral*)*, 2
og_bm_kernel *(*og_bm_spectral*)*, 2
og_bm_spectral, *2*, 2
og_center_data *(*og_process_from_fpca*)*, 13
og_component *(*og_predict*)*, 12
og_design_betas, 4, *7*, *9*
og_design_sampler, *4*, 5, *16*
og_fb_eigen, *3*, 6
og_fb_phi *(*og_fb_eigen*)*, 6
og_features *(*og_moments_all*)*, 10
og_fit, *2*, 7, *9*
og_fit_constrained, *2*, 8
og_fourier_phi *(*og_process_kl*)*, 14
og_gram *(*og_design_betas*)*, 4
og_hermite_features *(*og_multi_indices*)*, 11
og_information_profile, *2*
og_information_profile *(*og_design_betas*)*, 4
og_kernels, *2*, 10
og_moments_all, *4*, 10, *19*
og_multi_indices, *4*, *11*, 11, *19*
og_oracle_Kg *(*og_true_coefficients*)*, 19
og_predict, *2*, 12
og_process_bm, *2*, *6*
og_process_bm *(*og_process_kl*)*, 14
og_process_from_fpca, *2*, 13
og_process_kl, *2*, *4*, *7*, *9–11*, 14, *16*
og_scenario, 15, *18*
og_simulate, *2*, 16, *18*
og_simulate_bm, *2*, 17, *18*
og_simulation_design, *2*, *16*, 18
og_true_coefficients, 19
ogeom *(*ogeom-package*)*, 2
ogeom-package, 2

print.og_fit *(*og_fit*)*, 7